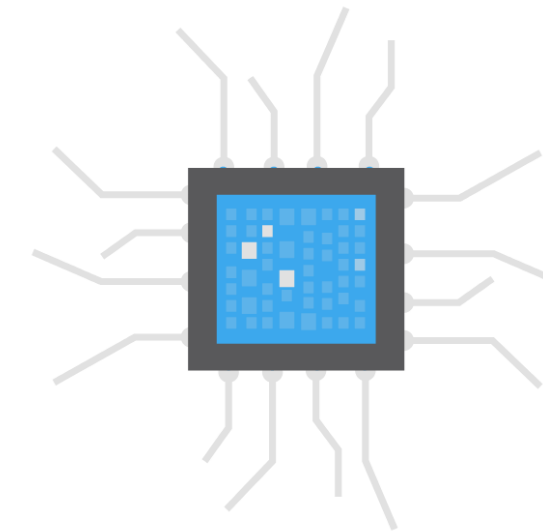


GPIOs

General Purpose Input | Output



GPIO

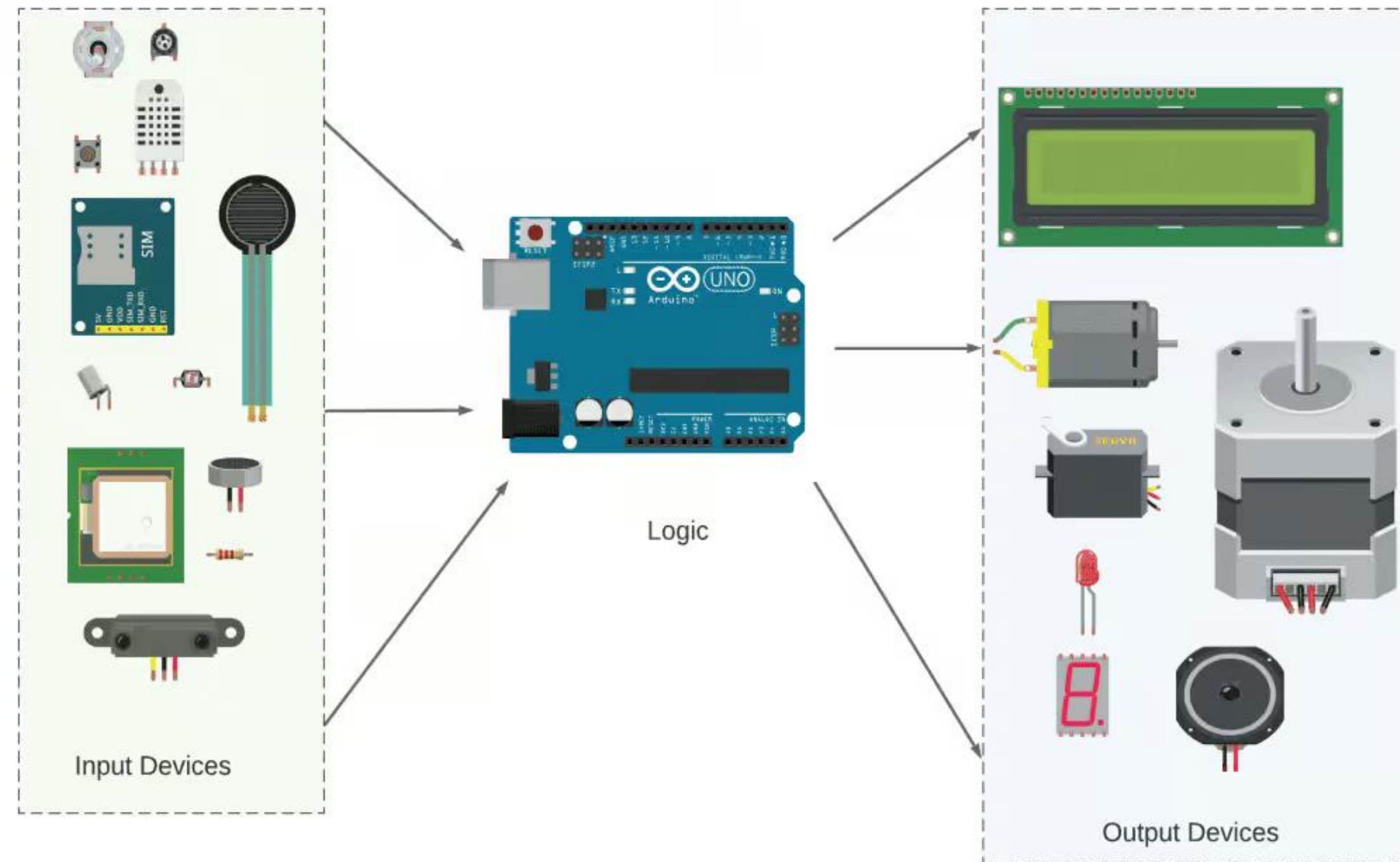
Introdução

- GPIOs são pinos de uso geral, agrupados em portais, que podem ser configurados como entrada ou saída.
- Para configurarmos quais pinos serão entrada/saída, utilizaremos registradores para cada portal: PB, PC e PD, cada um com 8 pinos.
- Para colocar informação (saída) ou ler informação (entrada) também utilizaremos registradores que acessam os pinos físicos do microcontrolador.

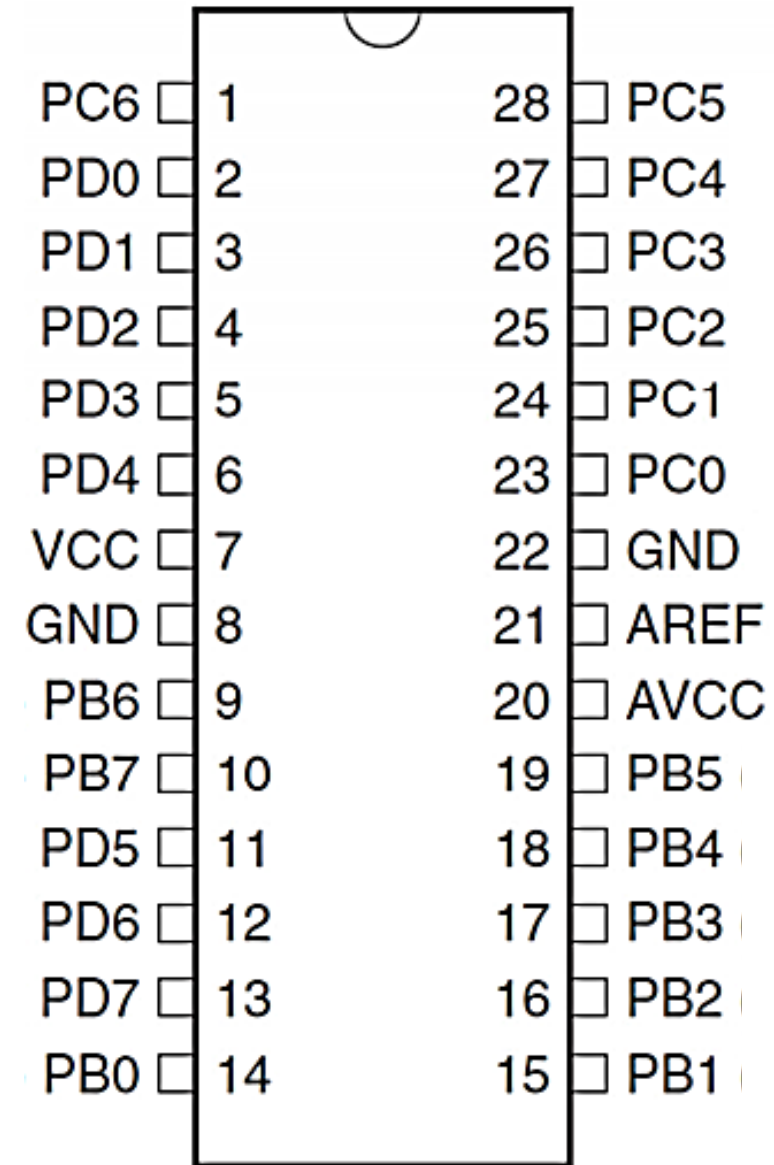


GPIO

- Componentes podem fornecer informação (entrada) ou serem controlados (saída).



GPIO



- O ATmega328p possui portais nomeados como B, C e D;
- Cada portal possui 8 pinos, que vão de Px0 a Px7;

Cada microcontrolador pode ter a nomenclatura que a **fabricante** definir. Exemplo:

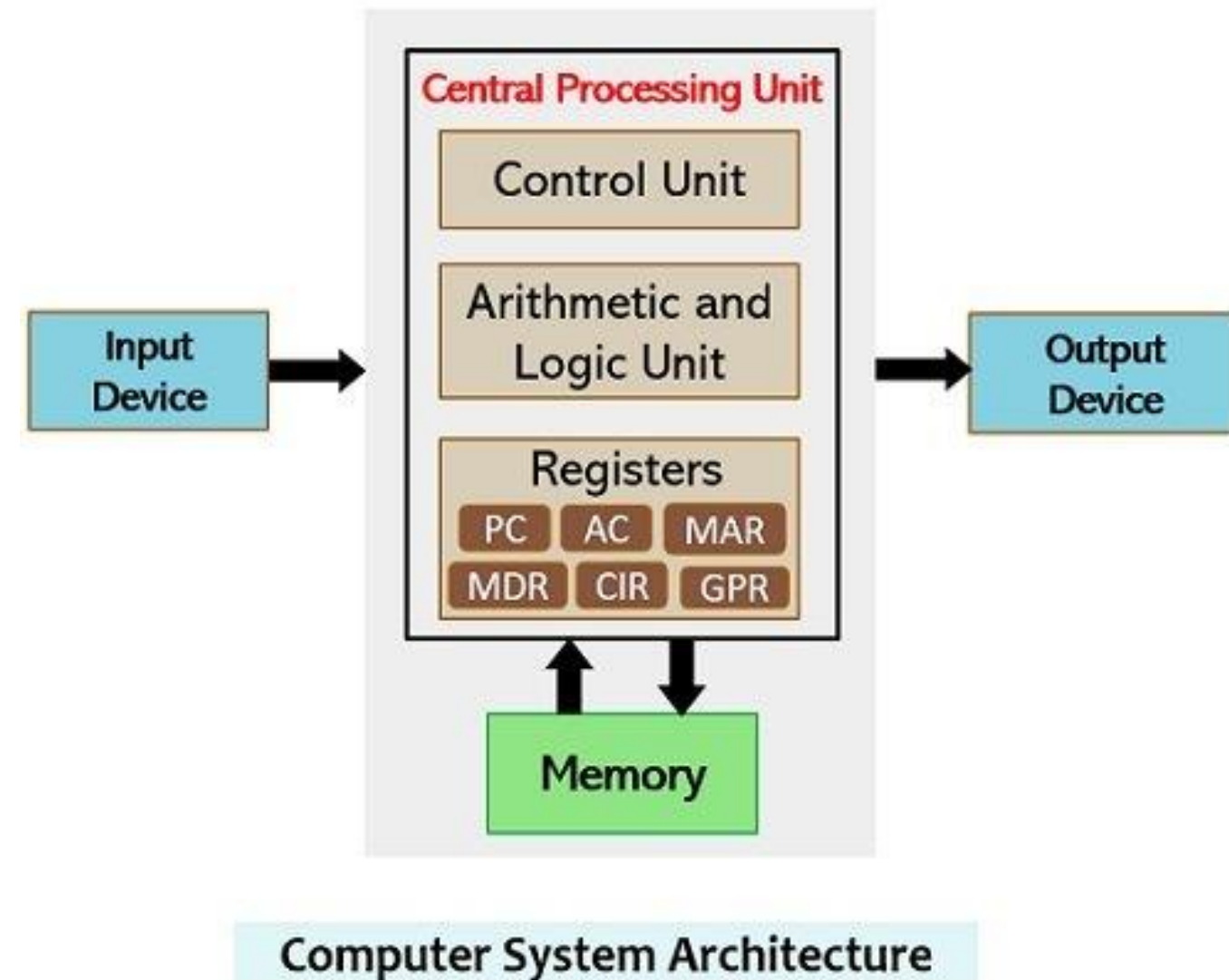
P1, P2, P3... PORTA, PORTB... PA, PB, PC



Registradores

Introdução

- Unidade de armazenamento que armazena dados temporários ou instruções.
- Ele é usado para realizar operações rápidas e manipular informações diretamente na CPU.
- Sua capacidade depende da arquitetura do microcontrolador.
ATMega328p -> Registradores de 8 bits



Registradores

Documentação

- Os registradores geralmente possuem bits exclusivos para cada configuração de hardware ou operação executada pela CPU.
- Todas informações são contidas nos *datasheets* dos microcontroladores.



13.4 Register Description

13.4.1 MCUCR – MCU Control Register

Bit	7	6	5	4	3	2	1	0
0x35 (0x55)	—	BODS	BODSE	PUD	—	—	IVSEL	IVCE
Read/Write	R	R	R	R/W	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

- Bit 4 – PUD: Pull-up Disable

When this bit is written to one, the pull-ups in the I/O ports are disabled even if the DDxn and PORTxn registers are configured to enable the pull-ups ({DDxn, PORTxn} = 0b01). See [Section 13.2.1 “Configuring the Pin” on page 59](#) for more details about this feature.

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

13.4.4 PINB – The Port B Input Pins Address

[illegible]

GPIO - Registradores

Estes são os registradores de controle utilizados na operação de GPIOs:

DDR_x (leitura/escrita): Registrador para configurar se os pinos serão entrada/saída.

PORT_x (leitura/escrita): Registrador para escrita de nível lógico (1/0) nos 8 pinos do portal.

PIN_x (leitura): Registrador para ler o nível lógico atual nos 8 pinos (entrada).





GPIO

Para trabalharmos com pinos de GPIO, trabalharemos na seguinte ordem:

- 1) Definir o pino como entrada ou saída (DDRxn);
 - *Ex.: **DDRB |= (1 « PB2);***
- 2) Quando for entrada, leremos o estado com (PINxn);
 - *Ex.: **if (PINB & (1«PB3))***
- 3) Quando for saída, mudaremos o estado com (PORTxn);
 - *Ex.: **PORTC |= (1«PC4);***



GPIO

Pull-Ups

- Uma porta do microcontrolador é como uma porta de um cômodo, ela pode estar **fechada** ou **aberta**.
- Mas como definimos o estado da porta quando ela está entreaberta?
- Quais mecanismos podemos utilizar para que, quando a porta não estiver aberta, ela esteja **com certeza fechada**?



GPIO

Solução – Uma mola de fechamento



E se instalarmos uma mola que garante que ao soltar a porta (quando ela não estiver aberta) ela esteja fechada?

Pinos eletrônicos possuem um problema parecido quando ficam em estado flutuante, conhecido como **Hi-Z (alta impedância)**.

Neste estado, ficam sensíveis a ruídos e interferências, gerando imprecisões em leituras.

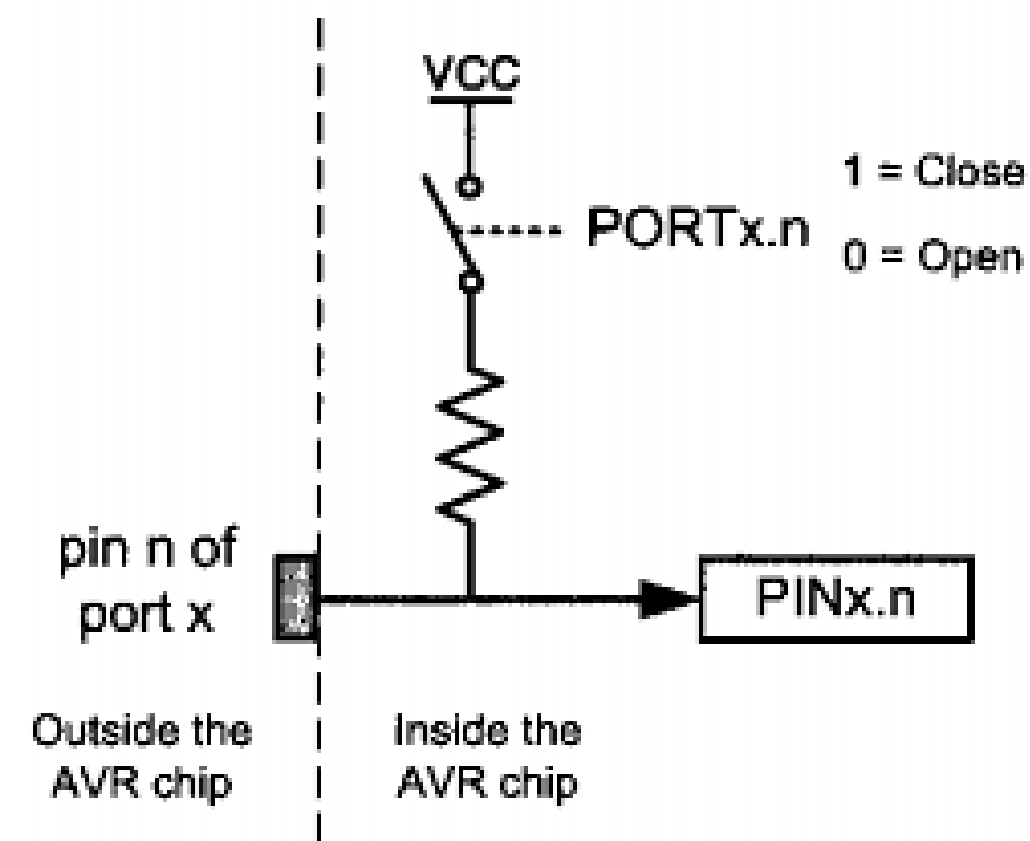


GPIO

Pull-Ups

Eletronicamente, podemos ativar um resistor de **pull-up interno**, que garante o nível lógico alto enquanto o pino estiver “livre”, atuando de forma similar à mola de uma porta real.

Para isso, o pino deve estar configurado como **entrada** com o DDRxn, e o PORTxn setado (em 1) no bit correspondente.





GPIO

Registradores

- **PORTx**: registrador de dados, usado para escrever nos pinos do PORTx.
- **DDRx**: registrador de direção, usado para definir se os pinos do PORTx são entrada ou saída.
- **PINx**: registrador de entrada, usado para ler o conteúdo dos pinos do PORTx.

Bits de controle dos pinos dos PORTs.

DDXn*	PORTXn	I/O	Pull-up	Comentário
0	0	Entrada	Não	Alta impedância (Hi-Z).
0	1	Entrada	Sim	PXn irá fornecer corrente se externamente for colocado em nível lógico 0.
1	0	Saída	Não	Saída em zero (drena corrente).
1	1	Saída	Não	Saída em nível alto (fornece corrente).

Manipulando Registradores

Uso de Registradores

** Para alterar o conteúdo dos registradores DDRx e PORTx é necessário realizar uma operação de escrita **de um byte completo**, mesmo que se deseje alterar apenas um dos bits.*

Agora se torna imprescindível o uso de **lógicas bit a bit e máscaras**.



Manipulando Registradores

Uso de Registradores

- | OU lógico bit a bit (usado para ativar bits , colocar em 1)
- & E lógico bit a bit (usado para limpar bits, colocar em 0)
- ^ OU EXCLUSIVO bit a bit (usado para trocar o estado dos bits)
- ~ complemento de 1 (1 vira 0, 0 vira 1)

$Nr \gg x$ O número é deslocado x bits para a direita

$Nr \ll x$ O número é deslocado x bits para a esquerda



Manipulando Registradores

Lógicas bit a bit

Lógica “e” bit a bit (&)

x: 10001101
y: 01010111
x & y: 00000101

Lógica “ou” bit a bit (|)

x: 10001101
y: 01010111
x | y: 11011111

Deslocamento para a esquerda

y = 1010
x = y << 1
Resulta
em: x = 0100

Operação “não” (~)

$\sim 0 = 1$
 $\sim 1 = 0$

Operação “ou-exclusivo” (^)

$0 \wedge 0 = 0$
 $0 \wedge 1 = 1$
 $1 \wedge 0 = 1$
 $1 \wedge 1 = 0$

Deslocamento para a direita

y = 1010
x = y >> 1
Resulta
em: x = 0101



Exemplos

1) Configurar apenas os pinos PB7 e PB6 como saída:

2) Setar apenas o PB6 para '1':

3) Resetar apenas o estado do pino PB6 para '0':



Exemplos

1) Configurar apenas os pinos PB7 e PB6 como saída:

`DDRB |= (1 « PB7) + (1 « PB6);`

2) Setar apenas o PB6 para '1':

`PORTB |= (1 « PB6);`

3) Resetar apenas o estado do pino PB6 para '0':

`PORTB &= ~(1 « PB6);`



Exemplo - Leitura

- Para ler um pino no registrador PIN, precisamos criar uma máscara para filtrar apenas o bit desejado, utilizando lógica E (&):

`if (PINC & (1 « PC2) == 0) → Este teste valida se PC2 == '0'`

`If (PINC & (1 « PC2) != 0) → Este teste valida se PC2 == '1'`

Cuidado: nunca utilize `(PINx & (1 « Pxn) == 1)` já que o resultado da operação E (&) nem sempre é 1 (decimal), ou 0b00000001.



Resumo

- Lógica **OU** (**|**) : seta um ou mais bits em '1' sem alterar o estado atual;
- Lógica **E com inverso** (**& ~**) : reseta um ou mais bits em '0' sem alterar o estado atual.
- Lógica **E** : utilizada em máscaras e retorna 0 ou o valor do bit da máscara.
 - Ex: $PINC \& (1 \ll PC0)$ vai retornar 0 ou $(1 \ll PC0)$





Operações

Para facilitar a manipulação dos registros, criaremos macros, utilizando `#define`.

Essas macros são similares a funções, onde especificaremos uma operação a ser executada, dado parâmetros inseridos na macro.



- **Ativação de bit, colocar em 1:**

```
#define set_bit(Y,bit_x) (Y|=(1<<bit_x))
```

onde $Y \mid= (1 \ll \text{bit_x})$ ou $Y = Y \mid (1 \ll \text{bit_x})$

Exemplo:

set_bit(PORTD,5)

$\text{PORTD} = \text{PORTD} \mid (1 \ll 5)$,

0bXXXXXXXX	(PORTD, x pode ser 0 ou 1)
$\mid 0\text{b00100000}$	($1 \ll 5$ é a máscara)
$\text{PORTD} = 0\text{bXX1XXXXX}$	(o bit 5 com certeza será 1)



▪ Limpeza de bit, colocar em 0:

```
#define clr_bit(Y,bit_x) (Y&=~(1<<bit_x))
```

onde $Y \&= \sim(1 \ll \text{bit_x})$ ou $Y = Y \& (\sim (1 \ll \text{bit_x}))$

Exemplo:

```
clr_bit(PORTB,2)
```

```
PORTB = PORTB & (~ (1<<2)) ,
```

0bXXXXXXXX	(PORTB, x pode ser 0 ou 1)
& 0b11111011	(~(1<<2) é a máscara)
PORTB = 0bXXXXX0XX	(o bit 2 com certeza será 0)



▪ **Troca o estado lógico de um bit, 0 para 1 ou 1 para 0:**

```
#define cpl_bit(Y,bit_x) (Y^=(1<<bit_x))
```

onde $Y \wedge = (1 \ll \text{bit_x})$ ou $Y = Y \wedge (1 \ll \text{bit_x})$

Exemplo:

cpl_bit(PORTC,3)

$\text{PORTC} = \text{PORTC} \wedge (1 \ll 3)$,

$0bXXXX1XXX$	(PORTC, x pode ser 0 ou 1)
$\wedge 0b00001000$	($1 \ll 3$ é a máscara)
$\text{PORTC} = 0bXXXX0XXX$	(o bit 3 será 0 se o bit a ser complementado for 1 e 1 se ele for 0)



▪ Leitura de um bit:

```
#define tst_bit(Y,bit_x) (Y&(1<<bit_x))
```

Exemplo:

tst_bit(PIND,4)

PIND & (1<<4) ,

0bxxxTxxxx	(PIND, x pode ser 0 ou 1)
& 0b00010000	(1<<4 é a máscara)
resultado = 0b000T0000	(o bit 4 terá o valor T, que será 0 ou 1)

Exercício

Considere o exemplo do portão eletrônico, feito com máquina de estados.

Defina pinos do microcontrolador ATmega328 (de sua escolha) para funcionar como entrada e saída do sistema, substituindo assim o uso de variáveis para simular os componentes.

Para cada pino, lembre-se de:

- Configurar como entrada ou saída (DDRx);
- Configurar os valores iniciais dos pinos do motor (MA/MF);
- Configurar resistores de pull-up para os sensores de fim de curso (abertura/fechamento);
- Fazer a leitura dos valores utilizando (PINx).

